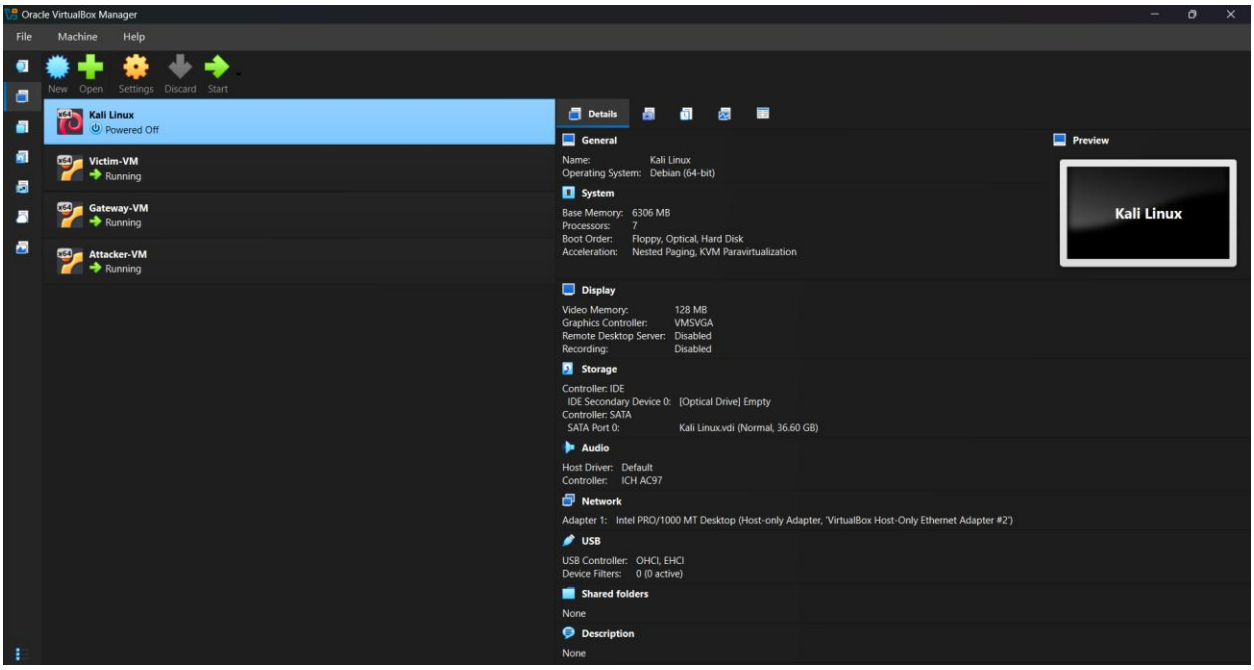

Figure 1. VirtualBox manager showing the three virtual machines used in the isolated lab: Victim-VM, Gateway-VM, and Attacker-VM.

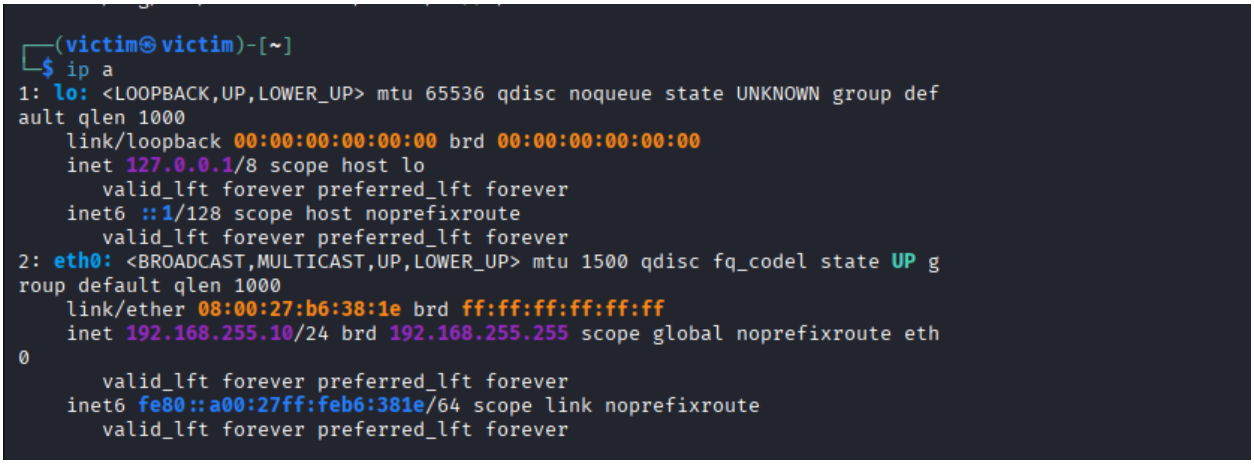

Figure 2. Victim VM network configuration showing interface eth0 with IPv4 address 192.168.255.10.

```
┌──(gateway㉿gateway)-[~]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group def
ault qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP g
roup default qlen 1000
    link/ether 08:00:27:ab:38:a2 brd ff:ff:ff:ff:ff:ff
    inet 192.168.255.1/24 brd 192.168.255.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:feab:38a2/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 3. Gateway VM network configuration showing interface eth0 with IPv4 address 192.168.255.1.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ ip a
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default qlen 1000
    link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00
    inet 127.0.0.1/8 scope host lo
       valid_lft forever preferred_lft forever
    inet6 ::1/128 scope host noprefixroute
       valid_lft forever preferred_lft forever
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group default qlen 1000
    link/ether 08:00:27:0b:7d:1b brd ff:ff:ff:ff:ff:ff
    inet 192.168.255.20/24 brd 192.168.255.255 scope global eth0
       valid_lft forever preferred_lft forever
    inet6 fe80::a00:27ff:fe0b:7d1b/64 scope link proto kernel_ll
       valid_lft forever preferred_lft forever
```

Figure 4. Attacker VM network configuration showing interface eth0 with IPv4 address 192.168.255.20.

```
[sudo] password for attacker:
IP=192.168.255.1, MAC=0a:00:27:00:00:3b
IP=192.168.255.10, MAC=08:00:27:b6:38:1e
2026-04-18 13:21:51,891 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-18 13:21:51,893 INFO sqlalchemy.engine.Engine SELECT hosts.id AS hosts_id, hosts.ip_address AS hosts_ip_address, hosts.mac_address AS hosts_mac_address, hosts.hostname AS hosts_hostname
FROM hosts
WHERE hosts.ip_address = ?
 LIMIT ? OFFSET ?
2026-04-18 13:21:51,894 INFO sqlalchemy.engine.Engine [generated in 0.00018s] ('192.168.255.1', 1, 0)
2026-04-18 13:21:51,895 INFO sqlalchemy.engine.Engine SELECT hosts.id AS hosts_id, hosts.ip_address AS hosts_ip_address, hosts.mac_address AS hosts_mac_address, hosts.hostname AS hosts_hostname
FROM hosts
WHERE hosts.ip_address = ?
 LIMIT ? OFFSET ?
2026-04-18 13:21:51,895 INFO sqlalchemy.engine.Engine [cached since 0.001457s ago] ('192.168.255.10', 1, 0)
2026-04-18 13:21:51,895 INFO sqlalchemy.engine.Engine COMMIT
Discovered hosts saved to database.
```

Figure 5. Attacker VM lab interface configuration confirming IPv4 address 192.168.255.20 before backend testing.

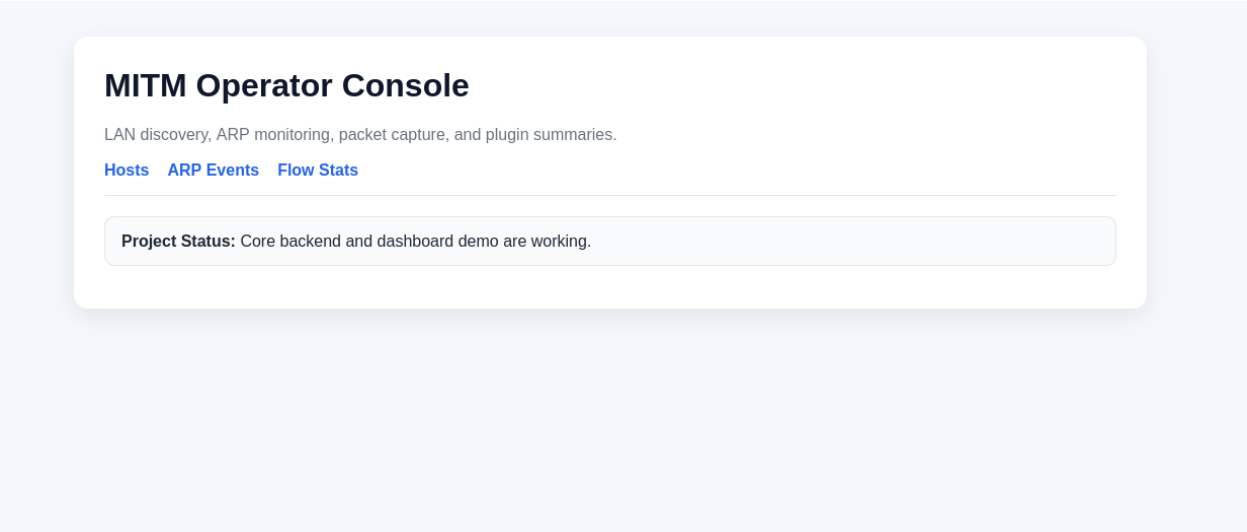

Figure 6. Host discovery and database-save output showing active LAN hosts identified and stored in SQLite.

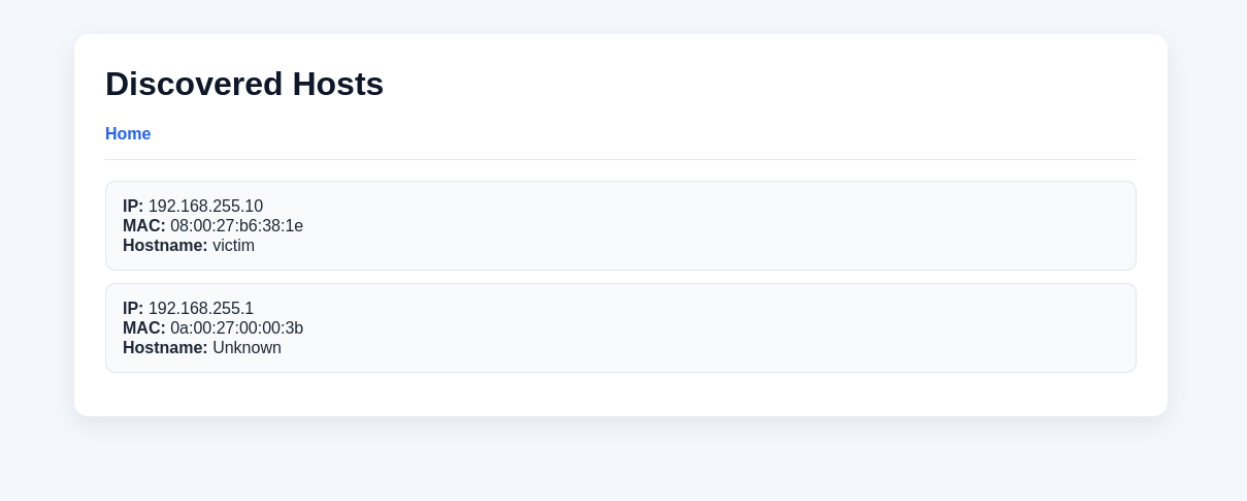

Figure 7. MITM Operator Console home dashboard.

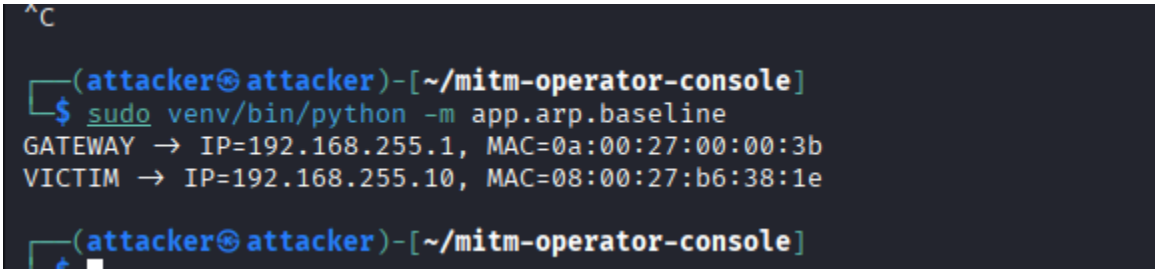

Figure 8. Discovered Hosts dashboard page listing the victim and gateway hosts.

```
┌──(victim㉿victim)-[~]
└─$ ping -c 2 192.168.255.20
PING 192.168.255.20 (192.168.255.20) 56(84) bytes of data.
64 bytes from 192.168.255.20: icmp_seq=1 ttl=64 time=1.74 ms
64 bytes from 192.168.255.20: icmp_seq=2 ttl=64 time=2.42 ms

--- 192.168.255.20 ping statistics ---
2 packets transmitted, 2 received, 0% packet loss, time 1001ms
rtt min/avg/max/mdev = 1.736/2.077/2.419/0.341 ms
```

Figure 9. ARP baseline capture showing the normal gateway and victim IP-to-MAC mappings.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ sudo venv/bin/python -m app.arp.monitor
Starting ARP monitor ... Press Ctrl+C to stop.
ARP Reply: 192.168.255.10 is-at 08:00:27:b6:38:1e → target 192.168.255.20
ARP Reply: 192.168.255.20 is-at 08:00:27:0b:7d:1b → target 192.168.255.10
ARP Reply: 192.168.255.20 is-at 08:00:27:0b:7d:1b → target 192.168.255.10
```

Figure 10. Victim-to-attacker connectivity validation using ICMP ping on the isolated LAN.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ sudo venv/bin/python -m app.arp.detect_changes
Starting ARP change detector ... Press Ctrl+C to stop.
[2026-04-18 13:28:32] Observed ARP Reply: 192.168.255.10 is-at 08:00:27:b6:38:1e
2026-04-18 13:28:32,899 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-18 13:28:32,900 INFO sqlalchemy.engine.Engine INSERT INTO arp_events (timestamp, source_ip, source_mac, event_type, message) VALUES (?, ?, ?, ?, ?)
2026-04-18 13:28:32,900 INFO sqlalchemy.engine.Engine [generated in 0.00012s] ('2026-04-18 13:28:32', '192.168.255.10', '08:00:27:b6:38:1e', 'OBSERVED', 'Observed ARP Reply: 192.168.255.10 is-at 08:00:27:b6:38:1e')
2026-04-18 13:28:32,901 INFO sqlalchemy.engine.Engine COMMIT
[2026-04-18 13:28:37] Observed ARP Reply: 192.168.255.20 is-at 08:00:27:0b:7d:1b
2026-04-18 13:28:37,987 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-18 13:28:37,988 INFO sqlalchemy.engine.Engine INSERT INTO arp_events (timestamp, source_ip, source_mac, event_type, message) VALUES (?, ?, ?, ?, ?)
2026-04-18 13:28:37,988 INFO sqlalchemy.engine.Engine [cached since 5.088s ago] ('2026-04-18 13:28:37', '192.168.255.20', '08:00:27:0b:7d:1b', 'OBSERVED', 'Observed ARP Reply: 192.168.255.20 is-at 08:00:27:0b:7d:1b')
2026-04-18 13:28:37,989 INFO sqlalchemy.engine.Engine COMMIT
```

Figure 11. Live ARP monitor output showing observed ARP reply traffic.

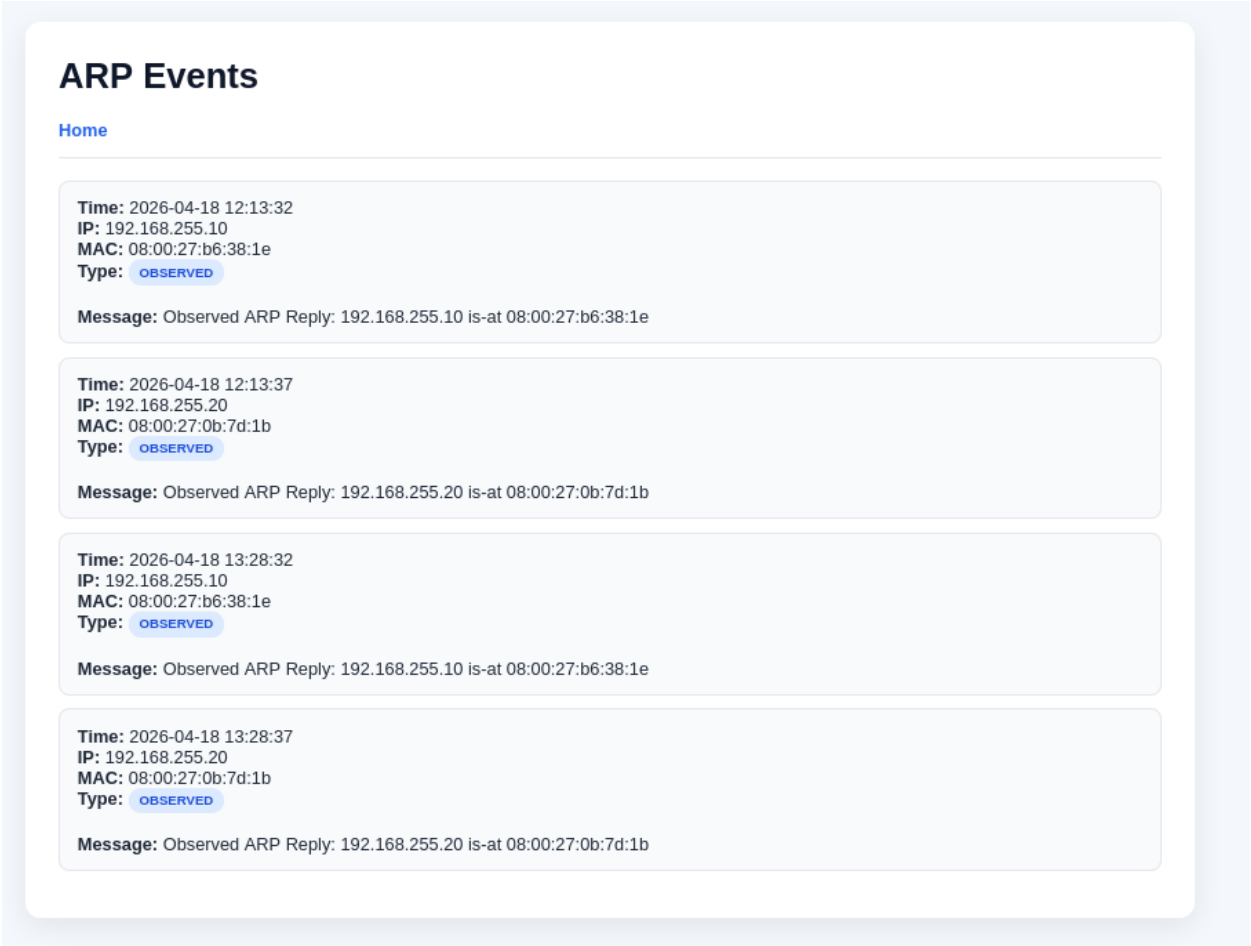

Figure 12. ARP change detector output with ARP observations being written to the database.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ python -m app.db.test_read
2026-04-18 13:30:35,900 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-18 13:30:35,901 INFO sqlalchemy.engine.Engine SELECT hosts.id AS hosts_id, hosts.ip_address AS hosts_ip_address, hosts.mac_address AS hosts_mac_address, hosts.hostname AS hosts_hostname
FROM hosts
2026-04-18 13:30:35,902 INFO sqlalchemy.engine.Engine [generated in 0.00031s] ()
ID=1, IP=192.168.255.10, MAC=08:00:27:b6:38:1e, HOSTNAME=victim
ID=2, IP=192.168.255.1, MAC=0a:00:27:00:00:3b, HOSTNAME=None
2026-04-18 13:30:35,902 INFO sqlalchemy.engine.Engine ROLLBACK
```

Figure 13. ARP Events dashboard page showing recorded ARP observations.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ python - << 'EOF'
from app.db.database import SessionLocal
from app.db.models import ArpEvent

db = SessionLocal()
events = db.query(ArpEvent).all()

for event in events:
    print(f"ID={event.id}, TIME={event.timestamp}, IP={event.source_ip}, MAC={event.source_mac}, TYPE={event.event_type}, MSG={event.message}")

db.close()
EOF
2026-04-18 13:32:42,766 INFO sqlalchemy.engine.Engine BEGIN (implicit)
2026-04-18 13:32:42,768 INFO sqlalchemy.engine.Engine SELECT arp_events.id AS arp_events_id, arp_events.timestamp AS arp_events_timestamp, arp_events.source_ip AS arp_events_source_ip, arp_events.source_mac AS arp_events_source_mac, ar
p_events.event_type AS arp_events_event_type, arp_events.message AS arp_events_message
FROM arp_events
2026-04-18 13:32:42,768 INFO sqlalchemy.engine.Engine [generated in 0.00016s] ()
ID=1, TIME=2026-04-18 12:13:32, IP=192.168.255.10, MAC=08:00:27:b6:38:1e, TYPE=OBSERVED, MSG=Observed ARP Reply: 192.168.255.10 is-at 08:00:27:b6:38:1e
ID=2, TIME=2026-04-18 12:13:37, IP=192.168.255.20, MAC=08:00:27:0b:7d:1b, TYPE=OBSERVED, MSG=Observed ARP Reply: 192.168.255.20 is-at 08:00:27:0b:7d:1b
ID=3, TIME=2026-04-18 13:28:32, IP=192.168.255.10, MAC=08:00:27:b6:38:1e, TYPE=OBSERVED, MSG=Observed ARP Reply: 192.168.255.10 is-at 08:00:27:b6:38:1e
ID=4, TIME=2026-04-18 13:28:37, IP=192.168.255.20, MAC=08:00:27:0b:7d:1b, TYPE=OBSERVED, MSG=Observed ARP Reply: 192.168.255.20 is-at 08:00:27:0b:7d:1b
2026-04-18 13:32:42,768 INFO sqlalchemy.engine.Engine ROLLBACK
```

Figure 14. Host table readback from SQLite confirming discovered hosts persisted in the database.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ sudo venv/bin/python -m app.capture.pcap_capture
Capturing 20 packets ...
Saved capture to data/pcaps/test_capture.pcap
```

Figure 15. ARP events table readback from SQLite confirming ARP observations persisted in the database.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ ls -lh data/pcaps/test_capture.pcap
-rw-r--r-- 1 root root 2.1K Apr 18 13:33 data/pcaps/test_capture.pcap
```

Figure 16. Packet capture session showing capture start and PCAP save location.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ python -m app.capture.read_pcap
Loaded 20 packets from data/pcaps/test_capture.pcap
```

Figure 17. File system listing confirming the saved PCAP artifact data/pcaps/test_capture.pcap.

```
┌──(attacker㉿attacker)-[~/mitm-operator-console]
└─$ python -m app.capture.summary
Summary for data/pcaps/test_capture.pcap

Protocol Counts:
ICMP: 16
ARP: 4

Top Flows:
192.168.255.10 → 192.168.255.20: 8
192.168.255.20 → 192.168.255.10: 8
```

Figure 18. PCAP readback output confirming 20 packets were loaded from the saved capture.

Figure 19. Protocol and flow summary generated from the saved PCAP.

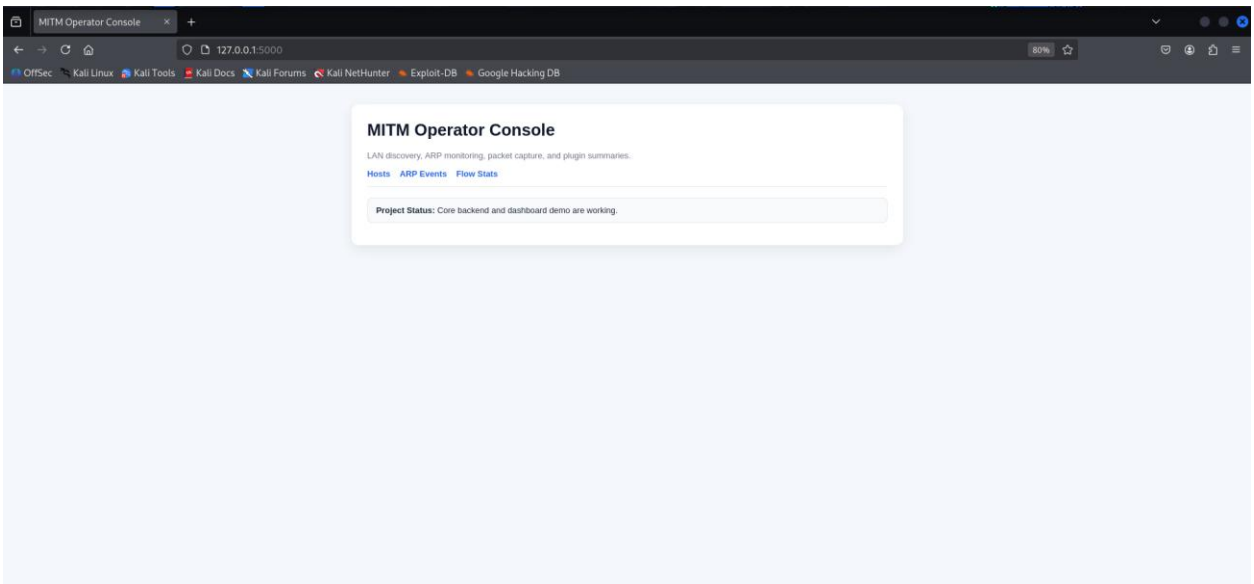

Figure 20. Flow Stats dashboard page visualizing protocol counts and top IP flows.

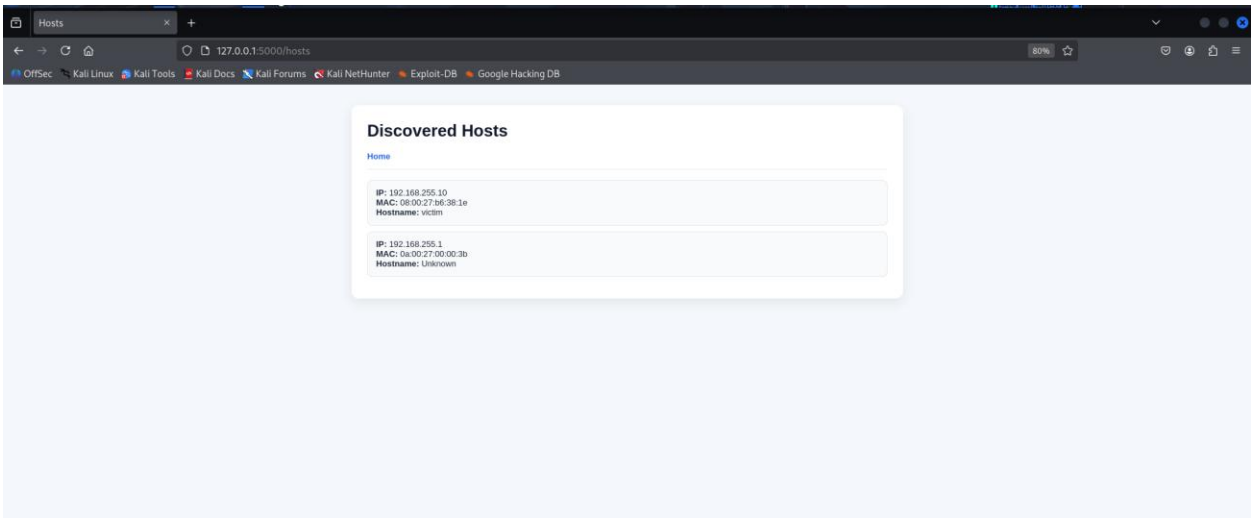

Figure 21. Styled MITM Operator Console home dashboard.

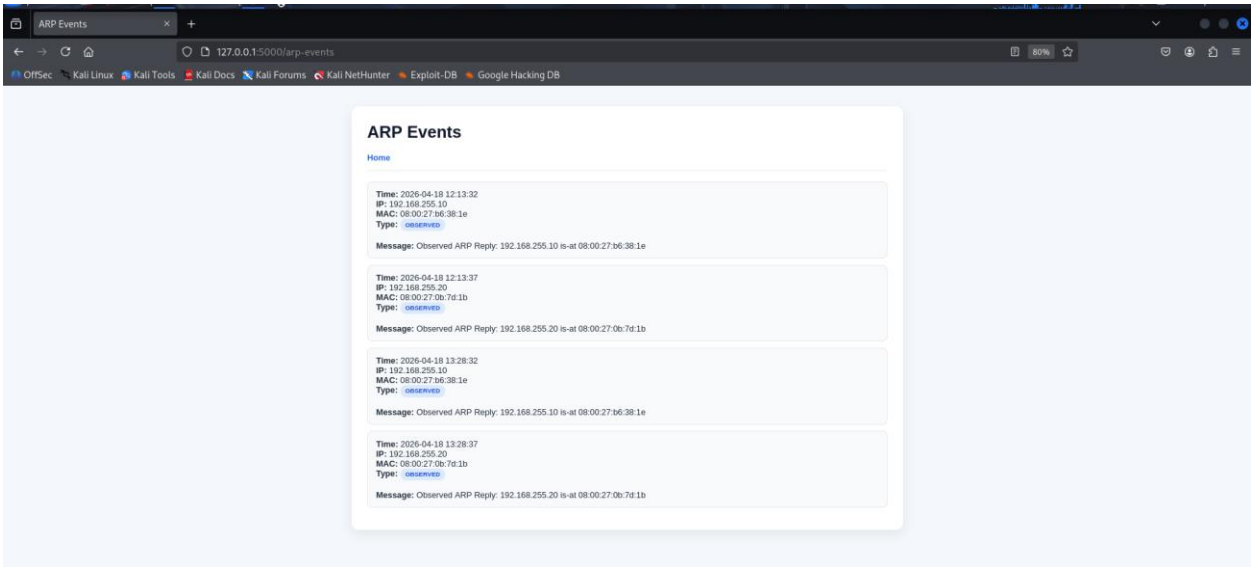

Figure 22. Styled Discovered Hosts dashboard page.

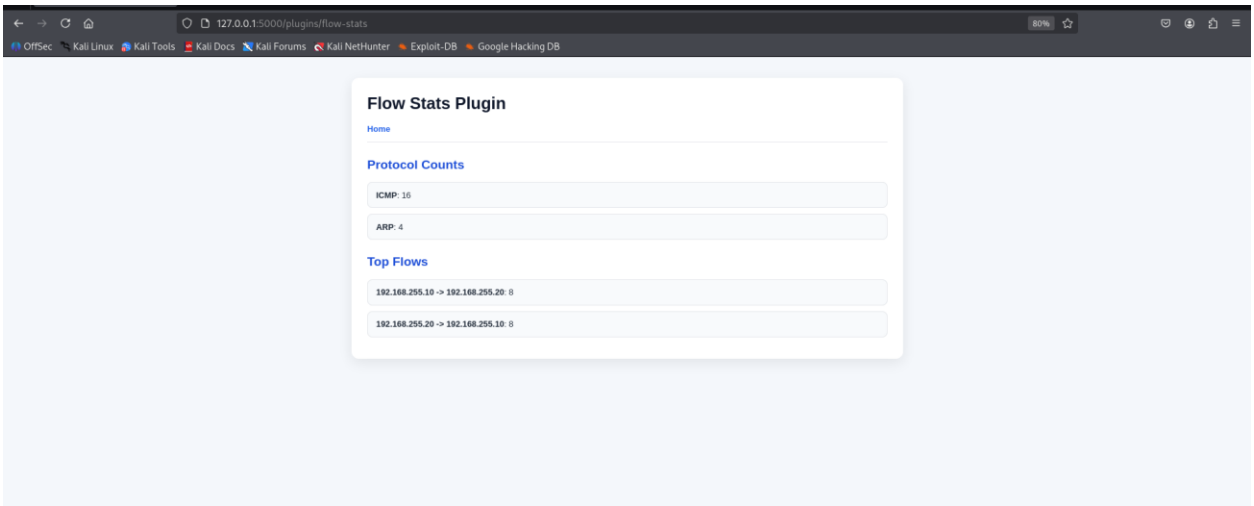

Figure 23. Styled ARP Events dashboard page.